ALGORITHMEN UND DATENSTRUKTUREN

Rangliste mit einem binären Suchbaum

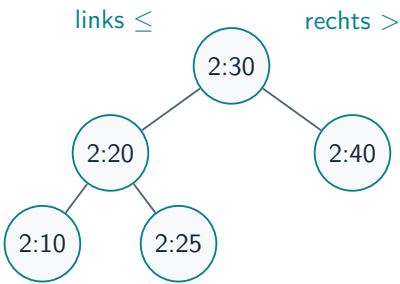
Aufgabe 3: Implementierung und Experiment

Name 1 & Name 2
ZHAW – Algorithmen und Datenstrukturen

3 166
Teilnehmer aus der CSV

Wie stark verändert die Einfügereihenfolge den Aufwand?

Suchbaum und sortierte Ausgabe



Beispielzeiten in Stunden:Minuten

Ordnung im Baum

`compareTo()` vergleicht die Laufzeiten. Gleiche Zeiten gehen nach links.

Inorder

Links, Knoten, rechts

2:10, 2:20, 2:25, 2:30, 2:40

Implementierung: Einfügen und Visitor

1. Teilnehmer einfügen

```
Tree<Competitor> tree =
    new SortedBinaryTree<>();

// Je gueltiger CSV-Zeile:
tree.add(new Competitor(
    0, name, time));
```

`createTree()` zerlegt die Zeilen, prüft das Format und füllt den Baum.

2. Rang und Text erzeugen

```
AtomicInteger rank =
    new AtomicInteger(1);
StringBuilder sb = new StringBuilder();

tree.traversal().inorder(c -> {
    c.setRank(rank.getAndIncrement());
    sb.append(c).append("\n");
});
```

Traversierung: Reihenfolge. **Visitor: Aktion je Teilnehmer.**

Ausschnitte aus `RankingTreeServer`; Ränge 1, 2, 3, ... auch bei Zeitgleichheit.

Laufzeit von Aufbau und Ausgabe

	Best Case	Average Case*	Worst Case
Baumhöhe h	$\Theta(\log n)$	erwartet $\Theta(\log n)$	$\Theta(n)$
Aufbau	$\Theta(n \log n)$	erwartet $\Theta(n \log n)$	$\Theta(n^2)$
Inorder-Ausgabe	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$

Einfügen kostet $O(h)$.

Die Summe der Einfügetiefen zählt die Vergleiche beim Aufbau.

Gesamt: Aufbau + Ausgabe

Speicher: $\Theta(n)$ für den Baum, zusätzlich $O(h)$ Rekursionsstack.

*Klassische Erwartung für verschiedene Schlüssel in gleichverteilt zufälliger Reihenfolge. CSV-Zeitgleichheiten untersuchen wir empirisch. Konstant grosse Zeitfelder; Ausgabetext linear in seiner Länge. Theorie: Sedgewick/Wayne, Binary Search Trees.

Experiment: vollständig und als Stichprobe

40 320

Alle 8! **Reihenfolgen**

Acht verschiedene Zeiten, über die sortierten CSV-Zeiten verteilt. Wir untersuchen jede ihrer Permutationen.

10 000

Zufallsreihenfolgen der ganzen CSV

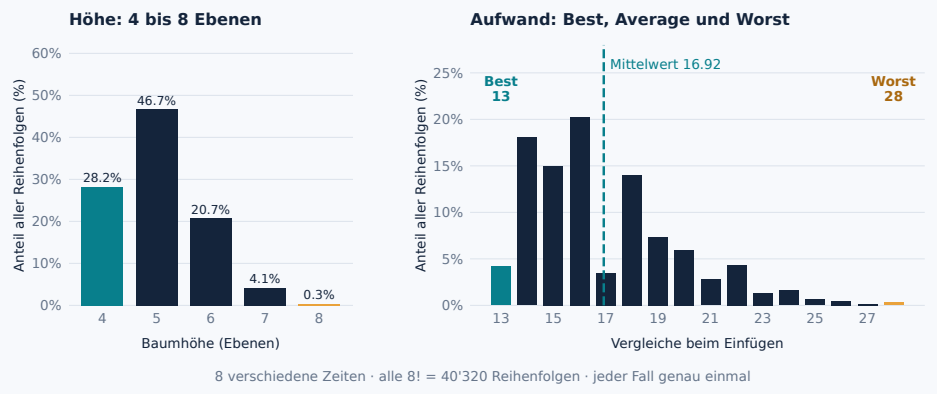
Je 3 166 Teilnehmer, fester Seed. Dazu Originaldatei, Median zuerst und sortierte Reihenfolgen.

Messgrössen: Höhe in Ebenen und Vergleiche beim Einfügen.

Alle Reihenfolgen der 3 166 markierten Datensätze wären rund 10^{9710}. Eine vollständige Untersuchung ist damit praktisch unmöglich.

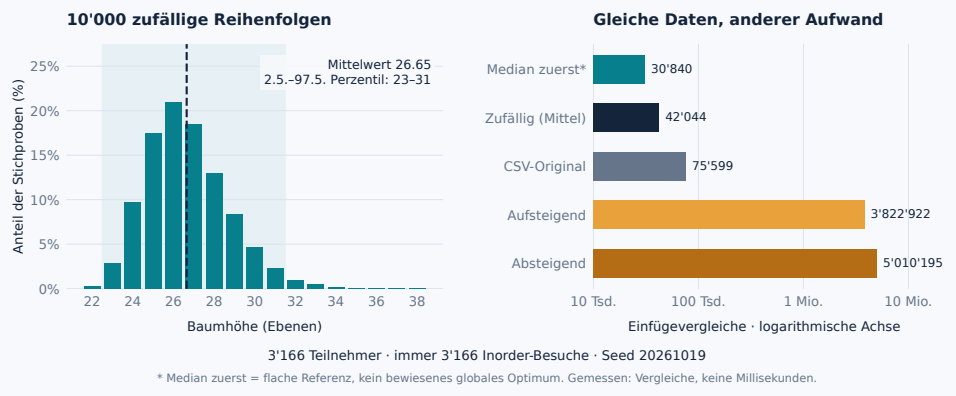
Messmodell: dieselbe Einfügeregel mit vorberechneten Sekundenwerten, gegen den Java-Baum geprüft. Keine Zeitmessung in Millisekunden. 2 459 verschiedene Zeiten; doppelte Werte bleiben erhalten. Seed: 20261019.

Acht Teilnehmer: die vollständige Verteilung



Einfügevergleiche: Best Case **13**, exakter Mittelwert **16,92**, Worst Case **28**.

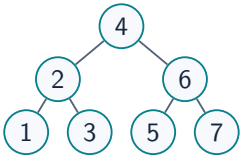
Die ganze CSV: Höhe und Einfügeaufwand



Höhe: Median zuerst **14**, Originaldatei **42**, absteigend **3 166**.

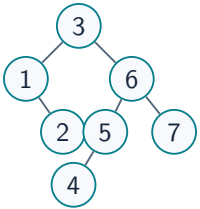
Warum diese Fälle entstehen

Flacher Baum



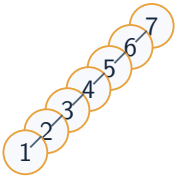
Mittlere Schlüssel zuerst teilen die Werte auf beide Seiten auf.

Zufällige Reihenfolge



Unterschiedliche Teilungen ergeben eine Streuung der Höhen.

Absteigende Reihenfolge



Jeder neue Wert folgt demselben Ast. Es entsteht eine Kette.

Die Einfügereihenfolge bestimmt die Baumform und damit den Aufwand.

Beispiele mit sieben unterschiedlichen Schlüsseln. Ein AVL- oder Rot-Schwarz-Baum würde seine Höhe selbst begrenzen.